

APPJULIA • REFERÊNCIA DE DADOS

Dossiê Julia

Como consultar e entender os dados do Julia — handoff técnico para agentes de IA se conectarem aos dois bancos, entenderem a integração Chat↔CRM↔Agentes e montarem relatórios/estatísticas.

Gerado por análise direta do código-fonte e verificação ao vivo em produção
c:\dev\appjulia\docs\data-handoff.md · 2026-07-30

Sumário

1. 1. Visão geral
2. 2. Como conectar
3. 3. Chaves de correlação entre os dois bancos
4. 4. Dicionário de dados — Banco Supabase
5. 5. Dicionário de dados — Banco externo (legado)
6. 6. Papéis, atendentes e equipe
7. 7. Como Chat, CRM e Agentes se integram
8. 8. Cookbook de relatórios prontos
9. 9. Riscos & cuidados antes de agir
10. 10. Mapa de arquivos — para ir mais a fundo

1. Visão geral

Julia é um SaaS multi-tenant para escritórios de advocacia: atendimento omnichannel via WhatsApp com um **agente de IA** chamado Julia, mais CRM, helpdesk, telefonia e integrações jurídicas (Advbox, DataJud, ZapSign). Cada escritório é um **tenant**, identificado por `client_id`.

Há **dois bancos de dados completamente separados**, e isso é a primeira coisa a internalizar antes de escrever qualquer query:

Banco A — Supabase (nativo)

Postgres gerenciado pelo Supabase. Contém o **Chat/Inbox** (maior domínio), o **CRM Builder** (kanban moderno), **Tickets/Helpdesk**, telemetria, telefonia Wavoip, billing. Acessível **diretamente** via client JS/HTTP (PostgREST) — inclusive com a chave anônima, porque o RLS é permissivo (ver seção 9).

Banco B — Postgres externo (legado)

Servidor Postgres gerenciado à parte (porta 25061, fora do Supabase). Contém `users`, `clients` (tenant real), `agents` (agente de IA), o **CRM clássico** (`crm_atendimento_*`), contratos, followup, campanhas de anúncio e permissões. **Só é acessível via a edge function `db-query`** — não há conexão direta possível para um agente externo.

Verificado ao vivo (consulta real em produção)

Contagens reais obtidas com a chave anônima do Supabase (`select('*',{count:'exact',head:true})`), confirmando que o RLS não bloqueia leitura:

Tabela (Supabase)	Linhas (produção)
<code>chat_messages</code>	1.122.914
<code>chat_conversations</code>	49.128
<code>chat_contacts</code>	46.021
<code>crm_deals</code>	5.936
<code>user_activity_log</code>	12.011
<code>queues</code>	91
<code>crm_boards</code>	88
<code>queue_agent_links</code>	35
<code>support_tickets</code>	18
<code>support_ticket_messages</code>	99

Implicação prática: qualquer relatório que precise cruzar dados de chat com dados de leads/agentes precisa buscar em **ambos os bancos separadamente e cruzar por telefone/código de agente em memória** (ver seção 3) — não existe uma query SQL única que atravesse os dois.

2. Como conectar

2.1 — Banco Supabase (leitura direta)

Use o client oficial `@supabase/supabase-js` com a URL e a chave anônima (publishable key) do projeto — ambas em `.env` na raiz do repo (`VITE_SUPABASE_URL` , `VITE_SUPABASE_PUBLISHABLE_KEY`). Como o RLS está com `USING (true)` em quase todas as tabelas, a chave anônima já lê tudo — não precisa de service role para relatórios de leitura.

```
// node - script ad-hoc de investigação
const { createClient } = require('@supabase/supabase-js');
const sb = createClient(process.env.SB_URL, process.env.SB_KEY); // .env: VITE_SUPABASE_URL /
VITE_SUPABASE_PUBLISHABLE_KEY

// contagem
const { count } = await sb.from('chat_conversations').select('*', { count: 'exact', head: true
});

// amostra + filtro por tenant
const { data } = await sb
  .from('chat_conversations')
  .select('id, contact_id, status, queue_id, cod_agent, client_id, opened_at')
  .eq('client_id', clientId)
  .order('opened_at', { ascending: false })
  .limit(50);
```

Dentro do próprio app React, o padrão é `import { supabase } from '@integrations/supabase/client'` e usar `supabase.from(tabela)` normalmente. Tabelas sem tipos gerados (ex.: telemetria/helpdesk mais novas) usam o cast `(supabase as any)` — padrão já usado em vários hooks do projeto.

2.2 — Banco externo (só via edge function)

Não existe outra porta de entrada. Toda leitura/escrita passa pela edge function `db-query` (`supabase/functions/db-query/index.ts`), invocada com um payload `{ action, data }`. As duas actions mais importantes para consultas ad-hoc:

```
// select genérico (equivalente a um SELECT simples numa tabela)
const { data } = await supabase.functions.invoke('db-query', {
  body: { action: 'select', table: 'crm_atendimento_cards', where: { cod_agent: '2026070001' },
  limit: 100 },
});

// raw: SQL arbitrário parametrizado — é assim que praticamente toda a
// analytics de CRM/Followup/Contratos/Campanhas do projeto funciona hoje
const { data } = await supabase.functions.invoke('db-query', {
  body: {
    action: 'raw',
    data: {
      query: `SELECT s.name, s.color, COUNT(c.id)::int AS count
              FROM crm_atendimento_stages s
              LEFT JOIN crm_atendimento_cards c ON s.id = c.stage_id AND c.cod_agent =
ANY($1::varchar[])
              WHERE s.is_active = true
              GROUP BY s.id, s.name, s.color, s.position
              ORDER BY s.position`,
      params: [['2026070001', '2026070002']],
    },
  },
});
```

⚠ **Atenção:** a action `raw` executa `sql.unsafe(query, params)` sem allowlist de tabela/verbo — é poderosa (é como o próprio frontend do Julia monta 90% dos relatórios hoje) mas também é a maior superfície de risco do sistema (ver seção 9). Use sempre com `params` parametrizados, nunca concatenando valor de usuário na string da query.

Wrapper de conveniência no frontend: `src/lib/externalDb.ts` (classe `ExternalDatabase`, singleton `externalDb`) — expõe `externalDb.raw(...)`, `externalDb.select(...)` e dezenas de métodos nomeados (`getUserAgents`, `getEffectiveClientId`, `getTeamByClient` etc.) que internamente chamam a action correspondente.

2.3 — Fora do app (agente autônomo sem sessão de usuário)

A edge function `db-query` não exige JWT de usuário (`verify_jwt=false` no `supabase/config.toml`) — ou seja, um agente com a URL do projeto e a chave anônima do Supabase já consegue invocá-la para leitura, do mesmo jeito do exemplo 2.1, só trocando o alvo:

```
const { data, error } = await sb.functions.invoke('db-query', {
  body: { action: 'raw', data: { query: 'SELECT COUNT(*) FROM agents WHERE status = true',
  params: [] } },
});
```

3. Chaves de correlação entre os dois bancos

Como os dois bancos são fisicamente separados, **qualquer relatório que precise combinar dados de chat (Supabase) com dados de CRM/agente (externo) precisa buscar dos dois lados e cruzar em memória**, usando uma destas chaves:

De	Para	Chave
<code>chat_contacts.phone</code> (Supabase)	<code>crm_atendimento_cards.whatsapp_number</code> (externo)	telefone normalizado BR 12/13 dígitos (9º dígito)
<code>chat_conversations.queue_id</code>	<code>cod_agent</code> do agente de IA	<code>queue_agent_links.cod_agent</code> (prioriza i
<code>crm_deals.custom_fields</code> (Supabase, CRM Builder)	<code>chat_conversations / chat_contacts</code>	JSONB <code>links.chat = {conversation_id, c</code> <code>contact_phone}</code>
<code>crm_deals.custom_fields</code>	<code>crm_atendimento_cards</code> (externo, "card Júlia")	JSONB <code>links.julia :</code> <code>whatsapp_number, cc</code>
<code>chat_contacts.cod_agent /</code> <code>chat_conversations.cod_agent /</code> <code>crm_deals.cod_agent</code> (Supabase)	<code>agents.cod_agent /</code> <code>crm_atendimento_cards.cod_agent /</code> <code>user_agents.cod_agent</code> (externo)	mesmo valor textual, s
usuário logado	<code>client_id</code> efetivo (tenant)	<code>COALESCE(users.cli</code> <code>pai.client_id,</code> <code>user_agents→agents.</code>
<code>chat_conversations.active_ticket_id</code>	<code>support_tickets.id</code>	mantido por trigger de <code>sync_conversation_</code>

3.1 — Normalização de telefone (a peça mais importante)

Números BR variam entre 12 e 13 dígitos (com/sem o 9º dígito após o DDD), e os dois bancos guardam formatos inconsistentes historicamente. **Sempre gere as variantes antes de comparar telefone entre os bancos** — nunca comparar string direta.

```
// src/lib/phoneVariants.ts – variantes usadas nas queries contra o banco externo
export function getBrPhoneVariants(raw: string | null | undefined): string[] {
  const d = (raw || '').replace(/\\D/g, '');
  if (!d) return [];
  const out = new Set<string>([d]);
  if (d.startsWith('55')) {
    const ddd = d.slice(2, 4);
    if (d.length === 13 && d[4] === '9' && /[6-9]/.test(d[5] ?? '')) {
      out.add(`55${ddd}${d.slice(5)}`); // remove o 9 extra
    } else if (d.length === 12 && /[6-9]/.test(d[4] ?? '')) {
      out.add(`55${ddd}9${d.slice(4)}`); // insere o 9 extra
    }
  }
  return [...out].filter(Boolean);
}
```

Uso típico em SQL (banco externo): `WHERE c.whatsapp_number = ANY($1::varchar[])` com `params: [getBrPhoneVariants(telefone)]`. No lado Supabase, o equivalente é `src/lib/phoneNormalize.ts` (`normalizeBrPhone / brPhoneVariants`) com `.in('phone', variants)`.

3.2 — Resolução do `client_id` efetivo

```
-- action get_effective_client_id (db-query) – cascata de herança de tenant
SELECT COALESCE(
  u.client_id,
  parent.client_id,
  (SELECT a.client_id FROM user_agents ua JOIN agents a ON a.id = ua.agent_id
   WHERE ua.user_id = u.id AND a.client_id IS NOT NULL LIMIT 1)
)::text AS client_id
FROM users u
LEFT JOIN users parent ON parent.id = u.user_id
WHERE u.id = $1
LIMIT 1
```

Ordem: **(1)** `users.client_id` próprio → **(2)** `client_id` do usuário pai (`users.user_id`) → **(3)** `client_id` do primeiro agente vinculado via `user_agents`. Papéis `time / advogado / comercial` nunca têm `client_id` próprio — sempre herdam do titular (ver seção 6).

4. Dicionário de dados — Banco Supabase

Colunas confirmadas ao vivo em produção (lendo uma linha real de cada tabela via `supabase.from(t).select('*').limit(1)` com a chave anônima) — não deduzidas de migration antiga.

chat_conversations

Uma conversa (ticket de atendimento) de um `chat_contacts` num canal. **49.128 linhas.**

```
id, contact_id, client_id, cod_agent, channel, status, protocol, assigned_to, department,
priority, tags, opened_at, first_response_at, closed_at, resolved_at, close_reason, close_note,
metadata, created_at, updated_at, queue_id, snoozed_until, snooze_reason, snoozed_by,
last_customer_message_at, last_message_from_me, active_ticket_id, active_ticket_number,
active_ticket_protocol, assigned_user_id
```

- `status`: `pending` | `open` | `resolved` | `closed`
- `channel`: `whatsapp_uazapi` | `whatsapp_waba` | `webchat` | `instagram`
- `priority`: `urgent` | `high` | `normal` | `low`
- `queue_id` → `queues.id` (é como se descobre o `cod_agent` responsável, ver seção 3)

chat_contacts

Um contato de WhatsApp/Instagram/WebChat, único por `(phone, client_id)`. **46.021 linhas.**

```
id, client_id, cod_agent, phone, name, avatar, is_group, is_archived, is_muted, unread_count,
last_message_at, last_message_text, created_at, updated_at, channel_type, channel_source,
remote_jid, history_backfilled, wa_name, wa_verified_name, wa_business, wa_status,
lead_full_name, lead_email, lead_personalid, profile_fetched_at, profile_source,
avatar_storage_path, avatar_source_url, avatar_source_hash, avatar_refreshed_at,
avatar_refresh_requested_at
```

chat_messages

Mensagem individual — maior tabela do sistema. **1.122.914 linhas.**

```
id, contact_id, client_id, message_id, text, type, from_me, status, media_url, file_name,
caption, reply_to, metadata, timestamp, created_at, channel_type, external_id, is_forwarded,
forwarded_score, raw_payload, conversation_id, internal_note, sender_name, note_type, edited_at
```

- `type`: `text`|`image`|`video`|`audio`|`ptt`|`document`|`location`|`contact`|`sticker`|`revoked`
- `status`: `sending`|`sent`|`delivered`|`read`|`failed` (nunca regride)
- `from_me=true` + `internal_note=true` = nota interna, invisível ao cliente

crm_deals / crm_boards (CRM Builder)

Kanban moderno, nativo do Supabase — **diferente** do CRM clássico da Julia (que vive no banco externo, seção 5).

```
-- crm_deals – 5.936 linhas
id, pipeline_id, board_id, cod_agent, title, description, value, currency, contact_name,
contact_phone, contact_email, priority, status, position, expected_close_date, custom_fields,
tags, assigned_to, stage_entered_at, created_at, updated_at, created_by, client_id, due_date,
updated_by, assigned_user_id
```

```
-- crm_boards – 88 linhas
id, cod_agent, name, description, icon, color, position, is_archived, settings, created_at,
updated_at, created_by, client_id
```

`crm_deals.status`: `open|won|lost|archived`. `custom_fields` (JSONB) guarda os vínculos com chat/Julia (seção 3).

queues / queue_agent_links

```
-- queues – 91 linhas
id, client_id, name, channel_type, hub, evo_url, evo_apikey, evo_instance, waba_id, waba_token,
waba_number_id, is_active, is_deleted, deleted_at, created_at, updated_at, phone_number,
phone_resolved_at, settings, waba_webhook_status, waba_webhook_last_error,
waba_webhook_subscribed_at
```

```
-- queue_agent_links – 35 linhas
id, queue_id, cod_agent, is_primary, created_at
```

`channel_type`: `uazapi|waba`. Constraint única: um `cod_agent` só pode ter **uma** fila `is_primary=true` no sistema inteiro.

support_tickets (Helpdesk)

```
-- 18 linhas
id, number, subject, description, status, priority, department_id, category_id,
requester_user_id, requester_client_id, requester_name, requester_email, requester_phone,
assigned_to, assigned_to_name, tags, conversation_id, contact_id, opened_at, first_response_at,
resolved_at, closed_at, reopened_count, sla_first_response_due_at, sla_resolution_due_at,
resolution_note, csat_score, csat_comment, csat_at, created_at, updated_at, metadata, protocol,
assigned_user_id
```

`status`: `open|pending|in_progress|waiting_customer|resolved|closed`. `conversation_id` vincula opcionalmente a uma conversa de chat (sincronizado por trigger, seção 3).

Outras tabelas Supabase relevantes

Tabela/família	Domínio
chat_conversation_history	Timeline/auditoria por conversa (<code>action</code> : assigned, reopened, closed, returned_to_queue...)
chat_sla_configs	SLA por prioridade/agente (FRT/NRT/TTR)
chat_csat_responses	Pesquisa de satisfação pós-atendimento
chat_crm_links	Vínculo redundante conversa↔CRM Builder (<code>external_system='crm_builder'</code>)
mv_user_chat_daily / mv_user_phone_daily / mv_user_sessions_daily	Materialized views de performance de atendente por dia (ver seção 6)
user_activity_log / user_device_log / user_performance_log	Telemetria de login/dispositivo/performance web
julia_orders / julia_plans	Billing SaaS (Mercado Pago/Asaas/InfinityPay)
wavoip_call_logs / phone_call_logs	CDR de chamadas (Wavoip WhatsApp / SIP)

Detalhe completo por domínio: [chat.md](#), [tickets-telemetry.md](#), [telephony-payments.md](#).

5. Dicionário de dados — Banco externo (legado)

Extraído lendo por completo `supabase/functions/db-query/index.ts` (~3600 linhas) e `src/lib/externalDb.ts` — não há acesso direto para conferir ao vivo, então esta é a fonte de verdade disponível.

● **Grafias "erradas" são o nome real — use exatamente assim:** `campaing_ads` (não `*campaign_ads*`) · `sing_document` (não `*sign_document*`) · `zapsing_doctoken` (não `*zapsign_doctoken*`) · `agents_plan.satus` (não `*status*`) · `view "vw_list_client-agents-users"` (tem hífen, exige aspas duplas em SQL). Corrigir esses nomes ao escrever queries evita erro de "relation does not exist".

users

Login/tenant. PK `id`. FK auto-referente `user_id` → `users.id` (usuário pai/titular de sub-usuários).

Colunas: `id`, `name`, `email`, `password (bcrypt)`, `role`, `cod_agent`, `client_id`, `user_id`, `evo_url`, `evo_instance`, `evo_apikey`, `hub`, `data_mask`, `remember_token`, `use_custom_permissions`, `is_active`, `status`, `queue_access ('all'|'specific')`.

```
-- action 'login', bcrypt no servidor
SELECT u.id, u.name, u.email, u.role, u.cod_agent,
       COALESCE(u.client_id, parent.client_id) as client_id,
       u.user_id, u.evo_url, u.evo_instance, u.evo_apikey, u.data_mask, u.hub, u.created_at,
       u.password, u.is_active,
       COALESCE(c.photo, pc.photo) as avatar
FROM users u
LEFT JOIN users parent ON parent.id = u.user_id
LEFT JOIN clients c ON c.id = u.client_id
LEFT JOIN clients pc ON pc.id = parent.client_id
WHERE u.email = $1
LIMIT 1
```

Senha nunca é validada no browser: hash bcrypt comparado dentro da edge function (`bcryptjs`); hashes legados `$2y$` (PHP) são normalizados para `$2a$` antes do compare.

clients

O tenant real (escritório). Colunas: `id`, `name`, `business_name`, `federal_id` (CNPJ/CPF), `email`, `phone`, `country`, `state`, `city`, `zip_code`, `street`, `street_number`, `complement`, `neighborhood`, `photo`, `created_at`, `updated_at`.

agents + agents_plan

O agente de IA WhatsApp. PK `id`; chave de negócio `cod_agent` (texto, formato `AAAAMM` +sequencial, ex. `2026070001`) — é **essa** a coluna usada em todo lugar (Supabase e externo), não o `id` numérico.

- **agents**: `id`, `cod_agent`, `client_id`, `user_id` (owner), `settings (jsonb)`, `prompt`, `is_closer`, `agent_plan_id`, `due_date`, `status (bool)`, `is_visibilided`, `hub ('uazapi'|'waba'|null)`, `evo_url`,

```
evo_instance, evo_apikey, waba_id, waba_token, waba_number_id, last_used, created_at,
updated_at
```

- **agents_plan**: id, name, "limit" (int leads/mês – nome reservado, sempre entre aspas), status (bool)

```
-- get_agent_details: ficha completa do agente
SELECT a.id, a.cod_agent::text as cod_agent, a.status, a.is_closer,
  CASE WHEN jsonb_typeof(a.settings)='string' THEN (a.settings #>> '{}')::jsonb ELSE a.settings
END as settings,
  a.prompt, a.due_date, a.created_at,
  c.id as client_id, c.name as client_name, c.business_name,
  ap.id as plan_id, ap.name as plan_name, ap."limit" as plan_limit,
  u.id as user_id, u.name as user_name, u.email as user_email,
  ua.can_edit_prompt, ua.can_edit_config,
  (SELECT COUNT(DISTINCT s.id) FROM sessions s
   WHERE s.agent_id = a.id
        AND EXISTS (SELECT 1 FROM log_messages lm WHERE lm.session_id = s.id
                     AND lm.created_at >= DATE_TRUNC('month', CURRENT_DATE))) as leads_received
FROM agents a
JOIN clients c ON c.id = a.client_id
LEFT JOIN agents_plan ap ON ap.id = a.agent_plan_id
LEFT JOIN users u ON u.id = a.user_id
LEFT JOIN user_agents ua ON ua.agent_id = a.id AND ua.user_id = a.user_id
WHERE a.id = $1 LIMIT 1
```

user_agents

N:N usuário↔agente. **agent_id NULL** = usuário só "monitora" aquele agente (não é dono). Colunas: id, user_id, agent_id, cod_agent, can_edit_prompt (bool), can_edit_config (bool), created_at.

sessions + log_messages

sessions = par único (agent_id, whatsapp_number) — representa se a Julia está "ligada" (**active**) para aquele lead. **log_messages** só é usada para **contar** leads/mês (session_id, created_at).

```
-- get_session_status
SELECT s.id, s.active, s.whatsapp_number::text, s.created_at, s.updated_at, a.cod_agent::text
FROM sessions s
JOIN agents a ON a.id = s.agent_id
WHERE s.whatsapp_number::text = ANY($1)
      AND a.cod_agent::text = $2::text
ORDER BY s.created_at DESC LIMIT 1
```

crm_atendimento_cards + crm_atendimento_stages + crm_atendimento_history

O **CRM clássico** da Julia — kanban de leads por telefone, é a fonte de quase todas as estatísticas de conversão hoje. Estágios reais vistos no código: *Análise de Caso*, *Negociação*, *Contrato em Curso*, *Contrato Assinado*, *Desqualificado*.

- **_stages**: id, name, color, position, is_active

- `_cards`: `id`, `helena_count_id`, `cod_agent`, `contact_name`, `whatsapp_number`, `business_name`, `stage_id`, `notes`, `owner_name`, `created_at`, `updated_at`, `stage_entered_at`
- `_history`: `id`, `card_id`, `from_stage_id`, `to_stage_id`, `changed_by`, `changed_at`, `notes`

```
-- useCRMDData.ts: listagem de cards com flag de contrato
SELECT c.id, c.helena_count_id, c.cod_agent, c.contact_name, c.whatsapp_number,
       c.business_name, c.stage_id, c.notes, c.created_at, c.updated_at, c.stage_entered_at,
       s.name as stage_name, s.color as stage_color, c.owner_name, a.owner_business_name,
       EXISTS (
         SELECT 1 FROM crm_atendimento_history h
         JOIN crm_atendimento_stages hs ON h.to_stage_id = hs.id
         WHERE h.card_id = c.id AND hs.name IN ('Contrato em Curso', 'Contrato Assinado')
       ) OR s.name IN ('Contrato em Curso', 'Contrato Assinado') as has_contract_history
FROM crm_atendimento_cards c
LEFT JOIN crm_atendimento_stages s ON c.stage_id = s.id
LEFT JOIN "vw_list_client-agents-users" a ON c.cod_agent = a.cod_agent::text
WHERE c.cod_agent = ANY($1::varchar[])
      AND (c.stage_entered_at AT TIME ZONE 'America/Sao_Paulo')::date >= $2::date
      AND (c.stage_entered_at AT TIME ZONE 'America/Sao_Paulo')::date <= $3::date
ORDER BY c.stage_entered_at DESC
```

campaing_ads

Leads originados de Meta/Google Ads. `campaign_data` (JSONB) tem as chaves `sourceID`, `title`, `sourceApp` (facebook/instagram/google), `phone`. Colunas: `id`, `cod_agent`, `session_id`, `type`, `campaign_data (jsonb)`, `created_at`.

modules / user_permissions / role_default_permissions / queue_members / module_embeds

Sistema de permissões, criado dinamicamente (as tabelas nascem via `CREATE TABLE IF NOT EXISTS` na action `init_permission_system`).

- `modules`: `id`, `code` (unique), `name`, `description`, `category` (principal|crm|agente|sistema|admin|financeiro), `is_active`, `display_order`, `icon`, `route`, `menu_group`, `is_menu_visible`, `module_type`
- `user_permissions`: `id`, `user_id`, `module_id`, `can_view`, `can_create`, `can_edit`, `can_delete` — único por (`user_id`, `module_id`)
- `role_default_permissions`: `id`, `role`, `module_id`, `can_view`, `can_create`, `can_edit`, `can_delete` — único por (`role`, `module_id`)

followup_config / followup_queue / followup_history

Cadências automáticas de reengajamento de leads inativos. `_queue`: `id`, `cod_agent`, `session_id` (=telefone), `step_number`, `send_date`, `state`, `history`, `name_client`, `hub`, `chat_memory`.

Estados: `state='SEND'` = em followup ativo · `state='STOP'` = parado (respondeu, foi atendido por humano, ou desistiu). **1 lead = múltiplas linhas históricas** — o estado atual é sempre `DISTINCT ON (cod_agent, session_id) ORDER BY send_date DESC`.

```
-- useFollowupData.ts: estado atual do followup por lead
SELECT DISTINCT ON (cod_agent, session_id)
  id, cod_agent, session_id, step_number, send_date, state, history, name_client, created_at,
  hub, chat_memory
FROM followup_queue
WHERE cod_agent = ANY($1::varchar[])
ORDER BY cod_agent, session_id, send_date DESC
```

sing_document (contratos)

Contratos gerados/assinados via ZapSign. Colunas: `cod_document`, `cod_agent`, `whatsapp_number`, `session_id`, `signer_name`, `signer_cpf`, `signer_uf`, `signer_cidade`, `signer_bairro`, `signer_endereco`, `signer_cep`, `document_case`, `zapsing_doctoken`, `resume_case`, `status_document`, `created_at`.

`status_document`: `CREATED` (gerado, não assinado) · `SIGNED` · `PENDING` · `CANCELLED` · `DELETED`.

Views agregadas (só via `raw`)

View	O que é	
<code>vw_painelv2_desempenho_julia</code>	1 linha por sessão/conversa Julia: <code>`cod_agent, name, business_name, client_id, perfil_agent (SDR`</code>	<code>CLOSER), session_id, total_msg, whatsapp, status_document, stage_entered_at`</code>
<code>vw_painelv2_desempenho_julia_all</code>	Variante sem deduplicação, para contagem total de conversas	
<code>vw_painelv2_desempenho_julia_contratos</code>	Junta a view acima com <code>sing_document</code> — 1 linha por contrato, com <code>data_contrato</code> , <code>data_assinatura</code> , <code>situacao</code>	
<code>vw_send_followup_queue_card</code>	Espelho de <code>followup_queue</code> com colunas extras <code>whatsapp</code> , <code>node_count</code>	
<code>vw_equipe</code>	Equipe com <code>client_id</code> já resolvido (herda do titular) — criada pela própria action <code>create_vw_equipe</code>	
<code>"vw_list_client-agents-users"</code>	<code>cod_agent</code> , <code>owner_name</code> , <code>owner_business_name</code> — usada em quase todo JOIN de CRM/Dashboard	

Tabelas Advbox (integração jurídica)

 **Nomes reais ≠ nomes intuitivos:** `advbox_integrations` (plural) · `advbox_notification_rules` · `advbox_lead_sync` (singular) — confirmados direto no código das edge functions `advbox-sync` / `advbox-notify` / `advbox-query`.

`advbox_integrations`, `advbox_notification_rules`, `advbox_processes_cache`, `advbox_notification_logs`, `advbox_client_queries`, `advbox_lead_sync` — cache de processos jurídicos sincronizados do Advbox, com notificação automática ao cliente por WhatsApp quando há movimentação.

6. Papéis, atendentes e equipe

```
// src/types/permissions.ts — union completa de papéis
export type AppRole = 'admin' | 'colaborador' | 'user' | 'time' | 'advogado' | 'comercial';
```

- **admin** — administração da Julia (equipe do próprio SaaS, vê todos os tenants).
- **user** / **colaborador** — titulares de um escritório; têm **client_id** próprio.
- **time** / **advogado** / **comercial** — atendentes/membros de equipe; **nunca têm client_id próprio**, herdam do titular via **users.user_id**.

Listar a equipe/atendentes de um **client_id**

```
-- action get_team_by_client, via view vw_equipe
SELECT v.id, v.name, v.email, v.role, v.client_id::text AS client_id, v.photo,
       u.user_id, u.created_at, u.remember_token, COALESCE(ac.cnt, 0)::int AS agents_count
FROM vw_equipe v
JOIN users u ON u.id = v.id
LEFT JOIN (SELECT user_id, COUNT(*)::int AS cnt FROM user_agents GROUP BY user_id) ac ON
ac.user_id = v.id
WHERE v.client_id = $1
ORDER BY v.name
```

Performance de atendentes (Supabase — chat/telefonia)

Combinação de **materialized views** + RPCs, tudo filtrado por **client_id** + **user_id**:

```
// useTeamPerformance.ts — 4 fontes combinadas
supabase.from('mv_user_sessions_daily')
  .select('user_id, user_name, day_brt, worked_seconds, sessions_count')
  .eq('client_id', clientIdNum).in('user_id', userIds).gte('day_brt', from).lte('day_brt', to);

supabase.from('mv_user_chat_daily')
  .select('user_name, user_id, day_brt, received, resolved, returned, transferred,
avg_handle_seconds')
  .eq('client_id', clientIdText).gte('day_brt', from).lte('day_brt', to);

supabase.from('mv_user_phone_daily')
  .select('user_id, day_brt, calls_total, calls_answered, calls_outbound, talk_seconds,
unique_numbers')
  .eq('client_id', clientIdText).in('user_id', userIds);

supabase.rpc('get_team_online_seconds_by_day', { p_user_ids, p_from, p_to });
```

Métricas derivadas: **resolution_rate** = **round(resolved/received*100)**; **occupancy_pct** = **round(talk_seconds/worked_seconds*100)**.

Conversas assumidas por um atendente

Não existe coluna "assigned_count" — deriva-se do log de eventos `chat_conversation_history` filtrando `action='assigned'`:

```
supabase.from('chat_conversation_history')
  .select('conversation_id, to_user_id, to_value, actor_name, created_at')
  .eq('action', 'assigned')
  .gte('created_at', from).lte('created_at', to)
  .or(`to_user_id.eq.${uid},and(to_user_id.is.null,to_value.ilike.%${name}%)`);
```

Desfechos classificados a partir de `chat_conversation_history.action`: `resolved`
(`resolved|closed|auto_resolved_queue_switch|manual_closed_for_new_conversation`), `returned`
(`auto_returned|returned_to_queue`), `transferred` (`assigned` saindo do usuário).

7. Como Chat, CRM e Agentes se integram

7.1 — Fila (Supabase) é a fonte de verdade do canal físico

Uma `queue` (Supabase) guarda as credenciais reais de conexão (uazapi ou WABA). A edge function `sync-queue-to-agent` empurra essas credenciais para o `agents` do banco externo, permitindo que o agente de IA legado envie/receba pelo mesmo canal físico da fila:

- `channel_type='uazapi'` → resolve `agentId` via action `get_agent_by_cod` → chama `update_agent_connection` com `{evo_url, evo_apikey, evo_instancia}`.
- `channel_type='waba'` → chama `update_agent_waba_connection` com `{wabaId, wabaToken, wabaNumberId}`.
- `webchat / instagram` → não sincroniza (a Julia/IA não atende esses canais hoje).

7.2 — CRM Builder ↔ Chat: vínculo por JSON, com sincronização bidirecional por trigger

Um `crm_deals` pode se vincular a uma conversa (`custom_fields.links.chat`) e/ou a um card do CRM clássico (`custom_fields.links.julia`):

```
{
  "source": "chat",
  "links": {
    "chat": { "conversation_id": "...", "contact_phone": "...", "contact_name": "..." },
    "julia": { "card_id": 123, "whatsapp_number": "...", "cod_agent": "...", "stage_id": 4,
    "stage_name": "..." }
  }
}
```

Sincronização bidirecional real (triggers de banco): `trg_sync_deal_to_conversation` e `trg_sync_conversation_to_deal` propagam `assigned_to` e `priority` nos dois sentidos (com guarda `pg_trigger_depth() <= 1` contra loop). Assumir a conversa no chat atualiza o responsável do card; mudar a prioridade no CRM Builder atualiza no chat. Funções `map_priority_chat_to_crm / map_priority_crm_to_chat` traduzem `medium ↔ normal`.

O card do CRM clássico (Julia) é só **espelhado como badge** no deal (nome/etapa/cor atualizados a cada 60s via `useJuliaCardPreview`) — **a etapa da Julia mudar NÃO move o card no kanban automaticamente.**

7.3 — Desativação da Julia (handoff humano) — só um gatilho real

⚠ **Regra importante para qualquer relatório de "intervenção humana":** a sessão da Julia (`sessions.active`) só é desligada quando um atendente **assume** ou **transfere manualmente** a conversa (função `disableJuliaOnAssignOrTransfer` em `WhatsAppDataContext.tsx`). Enviar uma mensagem manual **não** desliga a Julia por si só. Webhooks/echoes/automações também não desligam.

Ao assumir/transferir: **(1)** resolve `cod_agent` via `queue_agent_links` (primary) → **(2)** se houver sessão ativa, chama `update_session_status(active=false)` → **(3)** sempre dispara `n8n_execute-followup-stop`

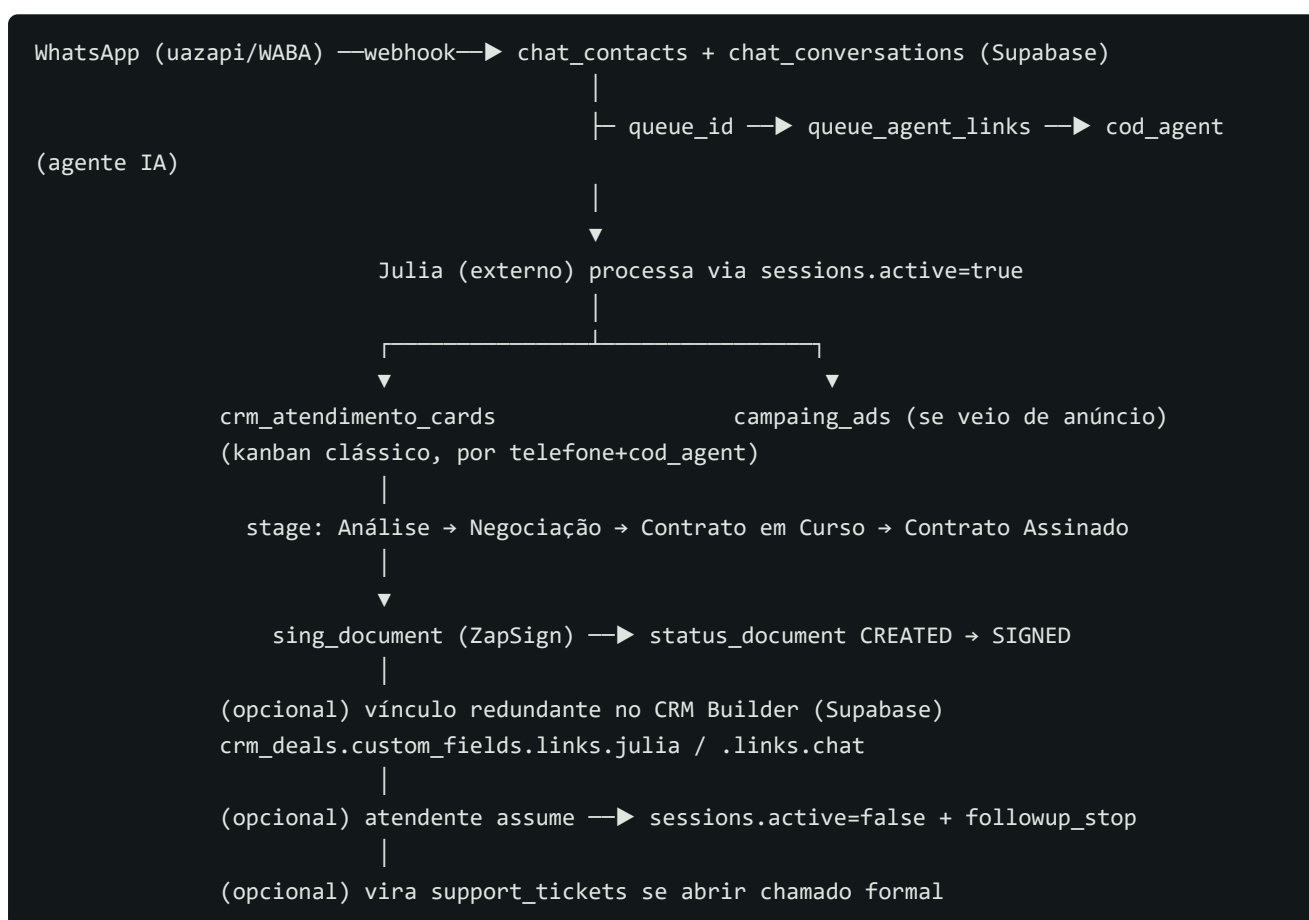
para parar followups pendentes daquele lead.

7.4 — Tickets ↔ Chat: vínculo mantido por trigger, não por código de app

Trigger `sync_conversation_active_ticket` (dispara em INSERT/UPDATE/DELETE de `support_tickets`): grava `chat_conversations.active_ticket_id/_number` quando o ticket está aberto (`open|pending|in_progress|waiting_customer`), e limpa quando o ticket é resolvido/fechado/deletado.

Isolamento por canal (regra corrigida em bug histórico): ao buscar conversa aberta para reaproveitar num webhook, **sempre** filtrar por `contact_id + client_id + queue_id + channel + status IN ('pending', 'open')` — sem isso, mensagens de canais diferentes (ex. WABA oficial vs UaZapi) se misturam no mesmo protocolo.

7.5 — Fluxo ponta-a-ponta



8. Cookbook de relatórios prontos

Queries reais já em produção no projeto — use como molde para relatórios novos.

8.1 — Funil de conversão do CRM clássico (por estágio)

```
-- via db-query/raw · useCRMStatistics.ts
SELECT s.id, s.name, s.color, s.position, COUNT(c.id)::int as count
FROM crm_atendimento_stages s
LEFT JOIN crm_atendimento_cards c ON s.id = c.stage_id
  AND c.cod_agent = ANY($1::varchar[])
  AND (c.stage_entered_at AT TIME ZONE 'America/Sao_Paulo')::date >= $2::date
  AND (c.stage_entered_at AT TIME ZONE 'America/Sao_Paulo')::date <= $3::date
WHERE s.is_active = true
GROUP BY s.id, s.name, s.color, s.position
ORDER BY s.position
```

8.2 — Performance por agente (taxa de qualificação e de contrato)

```
-- useCRMStatistics.ts
WITH julia_sessions AS (
  SELECT DISTINCT cod_agent::text as cod_agent, whatsapp::text as whatsapp
  FROM vw_painelv2_desempenho_julia
  WHERE cod_agent::text = ANY($1::varchar[])
    AND (created_at AT TIME ZONE 'America/Sao_Paulo')::date BETWEEN $2::date AND $3::date
),
qualified_stages AS (SELECT id FROM crm_atendimento_stages WHERE name IN
('Negociação','Contrato em Curso','Contrato Assinado')),
contract_stages AS (SELECT id FROM crm_atendimento_stages WHERE name IN ('Contrato em
Curso','Contrato Assinado'))
SELECT j.cod_agent, COALESCE(a.owner_name, j.cod_agent) as owner_name,
  COUNT(DISTINCT j.whatsapp)::int as total_leads,
  COUNT(DISTINCT CASE WHEN c.stage_id IN (SELECT id FROM qualified_stages) THEN j.whatsapp
END)::int as qualified_leads,
  (COUNT(DISTINCT CASE WHEN c.stage_id IN (SELECT id FROM qualified_stages) THEN j.whatsapp
END)::float
  / COUNT(DISTINCT j.whatsapp)) * 100 as qualified_rate,
  COUNT(DISTINCT CASE WHEN c.stage_id IN (SELECT id FROM contract_stages) THEN j.whatsapp
END)::int as contract_leads,
  AVG(EXTRACT(EPOCH FROM (COALESCE(c.updated_at, NOW()) - c.created_at)) / 86400) as
avg_time_days
FROM julia_sessions j
LEFT JOIN crm_atendimento_cards c ON c.whatsapp_number = j.whatsapp AND c.cod_agent =
j.cod_agent
LEFT JOIN "vw_list_client-agents-users" a ON j.cod_agent = a.cod_agent::text
GROUP BY j.cod_agent, a.owner_name
ORDER BY total_leads DESC
```

8.3 — MQL (leads qualificados) e conversões (Dashboard)

```
-- useDashboardData.ts - MQL
SELECT COUNT(*) as count
FROM crm_atendimento_cards c
JOIN crm_atendimento_stages s ON c.stage_id = s.id
WHERE s.name IN ('Negociação', 'Contrato em Curso', 'Contrato Assinado')
  AND c.cod_agent = ANY($1::varchar[])
  AND (c.stage_entered_at AT TIME ZONE 'America/Sao_Paulo')::date BETWEEN $2::date AND
$3::date;

-- Conversões (contratos gerados no período - fonte única de verdade)
SELECT COUNT(*) as count
FROM vw_painelv2_desempenho_julia_contratos
WHERE cod_agent::text = ANY($1::varchar[])
  AND (data_contrato AT TIME ZONE 'America/Sao_Paulo')::date BETWEEN $2::date AND $3::date
```

8.4 — Volume de chat (Supabase, direto)

```
const { count: abertas } = await supabase.from('chat_conversations')
  .select('*', { count: 'exact', head: true })
  .eq('client_id', clientId).in('status', ['pending', 'open']);

const { data: porFila } = await supabase.from('chat_conversations')
  .select('queue_id, status')
  .eq('client_id', clientId)
  .gte('opened_at', from).lte('opened_at', to);
// agregue por queue_id em memória, ou use um RPC dedicado se o volume for grande
```

8.5 — Campanhas de anúncio (funil + heatmap)

```
-- useCampanhasData.ts - leads por campanha
SELECT campaign_data->>'sourceID' as campaign_id, campaign_data->>'title' as campaign_title,
       COALESCE(campaign_data->>'sourceApp', 'outros') as platform,
       COUNT(*)::int as total_leads, MIN(created_at) as first_lead, MAX(created_at) as last_lead
FROM campaign_ads
WHERE cod_agent::text = ANY($1::varchar[])
      AND (created_at AT TIME ZONE 'America/Sao_Paulo')::date BETWEEN $2::date AND $3::date
      AND campaign_data->>'sourceID' IS NOT NULL
GROUP BY campaign_id, campaign_title, platform
ORDER BY total_leads DESC;

-- Heatmap dia-da-semana x hora
SELECT EXTRACT(DOW FROM created_at AT TIME ZONE 'America/Sao_Paulo')::int as day,
       EXTRACT(HOUR FROM created_at AT TIME ZONE 'America/Sao_Paulo')::int as hour,
       COUNT(*)::int as count
FROM campaign_ads
WHERE cod_agent::text = ANY($1::varchar[]) AND (created_at AT TIME ZONE
'America/Sao_Paulo')::date BETWEEN $2::date AND $3::date
GROUP BY day, hour ORDER BY day, hour
```

8.6 — Status de followup e contratos não assinados

```
-- Leads em followup ativo agora
SELECT DISTINCT ON (cod_agent, session_id) cod_agent, session_id, name_client, step_number,
send_date, state
FROM followup_queue
WHERE cod_agent = ANY($1::varchar[]) AND state = 'SEND'
ORDER BY cod_agent, session_id, send_date DESC;

-- Contratos gerados e não assinados (usado por contract-notifications-cron)
SELECT DISTINCT ON (s.cod_document) s.cod_document, s.whatsapp_number, s.signer_name,
       s.zapsing_doctoken, s.created_at
FROM sing_document s
WHERE s.cod_agent = $1 AND s.status_document = 'CREATED'
ORDER BY s.cod_document, s.created_at DESC
```

✓ **Padrão a replicar em qualquer relatório novo:** todas as queries acima seguem o mesmo padrão: **(1)** filtrar por `cod_agent = ANY($1::varchar[])` (lista de agentes visíveis ao usuário, resolvida via `externalDb.getCrmAgentsForUser`), **(2)** filtrar período convertendo para `America/Sao_Paulo` antes de comparar `::date`, **(3)** usar a coluna de data certa por domínio — `stage_entered_at` (CRM), `data_contrato` (contratos), `send_date` (followup), `created_at` (campanhas/sessões).

9. Riscos & cuidados antes de agir

● **SQL arbitrário exposto.** A action `raw` do `db-query` executa `sql.unsafe(query, params)` sem allowlist de tabela/verbo, e a função não exige JWT (`verify_jwt=false`). Na prática, qualquer chamador com a URL/chave anônima do projeto pode ler ou escrever qualquer coisa no banco externo. Isso **já é assim em produção hoje** — não é algo a "corrigir" ao gerar relatórios, mas é essencial nunca ampliar essa superfície (ex.: nunca montar uma query concatenando string vinda de input não confiável).

● **RLS do Supabase é permissiva.** Praticamente toda tabela tem policy `USING (true) WITH CHECK (true)` — confirmado ao vivo nesta análise (a chave anônima lê contagens de `chat_messages`, `crm_deals` etc. sem restrição). O isolamento por `client_id` é responsabilidade exclusiva do código da aplicação/das queries — **sempre filtrar por `client_id` explicitamente**, nunca assumir que o banco filtra por você.

⚠ **Não existe ID de lead único global.** A correlação entre chat, CRM clássico e agentes é sempre feita por **telefone normalizado + `cod_agent`** (seção 3), nunca por uma chave primária compartilhada. Gere sempre as variantes de telefone antes de comparar.

⚠ **Fuso horário.** Praticamente toda métrica de "período" no banco externo compara (coluna `AT TIME ZONE 'America/Sao_Paulo'`)::date — comparar direto contra `created_at` sem essa conversão desloca o dia em relatórios (o servidor guarda em UTC).

i **Nomenclatura com erros de grafia é intencional/histórica.** `campaing_ads`, `sing_document`, `zapsing_doctoken`, `agents_plan.satus` são os nomes **reais** das colunas/tabelas em produção — não "corrigir" ao escrever SQL.

Detalhe completo da postura de segurança do projeto: ver `data-layer.md` seções "Multi-tenancy" e "Achados-chave".

10. Mapa de arquivos — para ir mais a fundo

Precisa de...	Olhe em
Todas as ~120 actions do banco externo	supabase/functions/db-query/index.ts
Wrapper/cliente do banco externo	src/lib/externalDb.ts
Normalização de telefone BR	src/lib/phoneVariants.ts , src/lib/phoneNormalize.ts
Resolução de client_id	src/lib/resolveEffectiveClientId.ts
Queries de CRM clássico (funil, cards, stats)	src/pages/crm/hooks/useCRMData.ts , useCRMStatistics.ts
Queries de Dashboard/Estratégico	src/pages/dashboard/hooks/useDashboardData.ts , src/pages/estrategico/hooks/useJuliaData.ts
Followup	src/pages/agente/hooks/useFollowupData.ts
Contratos	src/pages/crm/hooks/useContractInfo.ts , supabase/functions/contract-notifications-cron/index.ts
Vínculo CRM Builder ↔ Chat	src/pages/crm-builder/hooks/useCardLinks.ts , useDealConversation.ts , useChatDealLink.ts
Fila ↔ agente IA	src/hooks/useQueueAgentLink.ts , supabase/functions/sync-queue-to-agent/index.ts
Handoff humano / desativação da Julia	src/contexts/WhatsAppDataContext.tsx (assignConversation , disableJuliaOnAssignOrTransfer)
Performance de equipe/atendentes	src/pages/equipe/hooks/useTeamPerformance.ts
Permissões e papéis	src/types/permissions.ts , action get_user_permissions
Documentação de arquitetura completa (já escrita)	../CLAUDE.md + demais arquivos em docs/
Notas técnicas específicas (triggers, bugs corrigidos)	mem/index.md + mem/features/ , mem/technical/

✓ **Resumo para o próximo agente:** se você está lendo isto para montar um relatório: **(1)** decida se o dado está no Supabase ou no banco externo (seção 4/5); **(2)** se precisar cruzar os dois, use telefone normalizado e/ou cod_agent (seção 3); **(3)** sempre filtre por client_id / cod_agent explicitamente — o banco não faz isso por você; **(4)** copie o padrão de uma query do cookbook (seção 8) em vez de escrever do zero.